

# Redes de Datos – Resumen 2do Parcial práctico (Prof. Baró)

**Fecha:** miércoles 21/10 (grupo Aguirre–Fassine), en el horario y aula de siempre. **Si llegás 10 minutos tarde, no rendís.** La teoría de Medín es el martes 27/10.

**Qué entra:** solo **Capa de Red**. Del Tanenbaum, cap. 5: **5.1, 5.2.1–5.2.5, 5.6.1–5.6.2, 5.6.4, 5.6.6–5.6.7**. Además, el apunte y la práctica de máscaras de Baró, y su guía de estudio. **No entran:** congestión, MPLS, broadcast/multicast/anycast, jerarquías, móviles, ad-hoc.

**Cómo leer este resumen:** todo sale de la wiki ( RD.md , Unidad 2) y de las fuentes de Baró, **salvo** lo marcado con [Tanenbaum]. Eso lo completé con conocimiento general del libro, porque ninguna de tus fuentes lo desarrolla. Son las preguntas de ruteo de la guía sobre tabla de ruteo, ruta default, SA, IGP, áreas, LSP y grafo. Si conseguís el PDF del libro (Baró lo ofrece por mail), conviene confirmarlo.

**Prioridad:** (1) **subnetting**, que es la parte práctica; (2) la **guía con preguntas y respuestas** (sección 7); (3) el header IP, ARP e ICMP; (4) ruteo, OSPF y BGP.

## CONTENIDO

- |                                                                      |                                                             |
|----------------------------------------------------------------------|-------------------------------------------------------------|
| 1 Servicios de la capa de red (Tanenbaum 5.1)                        | 5 Protocolos de control (Tanenbaum 5.6.4) y comandos de red |
| 2 Ruteo (Tanenbaum 5.2.1–5.2.5)                                      | 6 OSPF y BGP (Tanenbaum 5.6.6–5.6.7)                        |
| 3 Protocolo IP (Tanenbaum 5.6.1)                                     | 7 Guía de estudio de Baró — preguntas y respuestas          |
| 4 Direcciones IP y subnetting (Tanenbaum 5.6.2) — el núcleo práctico | 8 Ejercicios para practicar (con respuestas)                |
|                                                                      | Datos de memoria                                            |

## 1. Servicios de la capa de red (Tanenbaum 5.1)

La capa 3 lleva **paquetes** del origen al destino final atravesando varias redes. Su elemento principal es el **router**, que guarda el paquete, mira la IP destino y lo despacha por la línea de salida que indica su tabla (*store-and-forward*).

|                                  | SIN CONEXIÓN (DATAGRAMAS)                             | ORIENTADO A CONEXIÓN (CIRCUITOS VIRTUALES) |
|----------------------------------|-------------------------------------------------------|--------------------------------------------|
| Establecimiento                  | No hace falta                                         | Primero se arma el CV                      |
| Qué lleva cada paquete           | IP origen y destino completas                         | Solo el n° de CV                           |
| Estado en el router              | Ninguno                                               | Una entrada de tabla por cada CV           |
| Ruta                             | Independiente por paquete; pueden llegar desordenados | Todos por la misma ruta                    |
| Si cae un router                 | Se rutea por otro lado                                | Se cortan los CV que pasaban por él        |
| Calidad de servicio y congestión | Difícil                                               | Fácil: se reservan recursos al armar el CV |
| Ejemplo                          | IP (Internet)                                         | MPLS, ATM                                  |

**IP es sin conexión y no confiable ("best effort"):** los datagramas pueden perderse, duplicarse o llegar desordenados. La confiabilidad la pone la capa de transporte (TCP).

## 2. Ruteo (Tanenbaum 5.2.1-5.2.5)

---

### Conceptos base

- **Ruteo [Tanenbaum]:** es decidir **por qué línea de salida** se reenvía cada paquete para que llegue a destino. Se distinguen dos tareas: el **reenvío** (*forwarding*), que es consultar la tabla para cada paquete que llega, y el **ruteo** propiamente dicho, que es **armar y actualizar la tabla** con un **algoritmo de ruteo**.
- **Tabla de ruteo [Tanenbaum]:** una fila por destino (red/prefijo) con la **línea de salida o próximo salto** y la **métrica** de esa ruta. **La arma cada router:** con un algoritmo dinámico, intercambiando información con los demás (RIP, OSPF, BGP), o la carga a mano el administrador (ruteo estático).
- **Ruta default [Tanenbaum]:** la entrada que se usa **cuando el destino no coincide con ninguna otra** ( $0.0.0.0/0$ ). Existe porque un router no puede conocer todas las redes de Internet: lo desconocido se manda "hacia arriba", al router del ISP o de salida. Un host hace lo mismo con su **gateway** (puerta de enlace).
- **Estático vs dinámico:** las rutas **estáticas** (no adaptativas) se configuran a mano y no reaccionan a cambios. Las **dinámicas** (adaptativas) se recalculan solas cuando cambia la topología o el tráfico; son las que usan los routers de Internet.
- **Métrica:** es el **costo** asignado a un enlace o a una ruta, que sirve para comparar caminos y elegir el mejor. Ejemplos: **cantidad de saltos** (RIP), **retardo**, **ancho de banda** (OSPF: costo inversamente proporcional a la velocidad,  $1 \text{ Gbps} \rightarrow 1$  y  $100 \text{ Mbps} \rightarrow 10$ ), carga, distancia, costo económico.
- **Principio de optimalidad (5.2.1):** si J está en la ruta óptima de I a K, entonces la ruta óptima de J a K va por ese mismo camino. Por eso el conjunto de rutas óptimas hacia un destino forma un árbol: el **sink tree** (árbol sumidero). No tiene bucles, así que todo paquete llega en una cantidad finita de saltos.

### Algoritmos

- **Camino más corto, Dijkstra (5.2.2):** cada nodo se etiqueta con su distancia al origen. Se elige el nodo provisorio de menor etiqueta, se lo hace permanente y desde él se relajan sus vecinos. Ejemplo del teórico: la ruta  $A \rightarrow D$  más corta es ABEFHD, con costo 10.
- **Inundación (flooding, 5.2.3):** cada paquete se reenvía **por todas las líneas menos por la que llegó**. Para que no se multiplique sin fin se usan dos mecanismos: un **contador de saltos** en el header, que baja en cada router y descarta el paquete al llegar a 0, y un **número de secuencia** por origen para descartar duplicados. Es muy robusta (si existe un camino, lo encuentra), siempre encuentra también el camino más corto y sirve de base para otros algoritmos: estado de enlace la usa para distribuir sus paquetes. Su costo es mucho tráfico duplicado.

**Vector distancia (5.2.4), o Bellman-Ford:** cada router tiene una tabla, el "vector", con la **mejor distancia conocida a cada destino y por qué línea**. Periódicamente **la intercambia solo con sus vecinos** y recalcula: distancia al destino = mínimo, sobre cada vecino, de (distancia que informa ese vecino + costo hasta ese vecino). Así funcionaban ARPANET y luego **RIP**.

- Ejemplo del teórico: J tiene vecinos A (8), I (10), H (12) y K (6). Para llegar a G conviene ir **por H, con 18**.
- **Cuenta a infinito:** las buenas noticias se propagan rápido; las malas, lento. Cuando cae un enlace, los vecinos se siguen informando rutas viejas y suman de a 1. Se mitiga fijando un "infinito" chico (en RIP 16 saltos).

**Estado de enlace (5.2.5):** lo usan **OSPF** e **IS-IS**. Los **5 pasos** son pregunta de la guía:

1. **Descubrir a los vecinos** y sus direcciones, con paquetes **HELLO**.
  2. **Medir el costo** (la métrica) a cada vecino.
  3. **Armar un paquete de estado de enlace** (LSP) con lo aprendido.
  4. **Enviarlo a todos los routers** por inundación, y recibir los de todos los demás.
  5. **Calcular la ruta más corta** a cada router con **Dijkstra**.
- **Composición del LSP [Tanenbaum]:** la **identidad del emisor**, un **número de secuencia** (para descartar duplicados y paquetes viejos), una **edad** (se decrementa y el paquete expira al llegar a 0) y la **lista de vecinos con el costo a cada uno**.
  - **El grafo [Tanenbaum]:** con los LSP de todos, **cada router arma el grafo completo de la topología**: nodos = routers, aristas = enlaces con su costo. Sobre ese grafo corre Dijkstra **localmente** para obtener su sink tree y llenar su tabla. Sirve para que cada router calcule rutas óptimas por su cuenta, con información completa.

### Vector distancia vs estado de enlace (pregunta de la guía)

|                        | VECTOR DISTANCIA (RIP)                 | ESTADO DE ENLACE (OSPF)                            |
|------------------------|----------------------------------------|----------------------------------------------------|
| Qué informa            | <b>Toda su tabla</b> (cómo ve a todos) | <b>Solo sus enlaces</b> (a sus vecinos y su costo) |
| A quién                | <b>Solo a sus vecinos</b>              | <b>A todos los routers</b> (por inundación)        |
| Qué conoce cada router | Distancias, no la topología            | <b>La topología completa</b> (el grafo)            |
| Cálculo                | Bellman-Ford distribuido               | Dijkstra local                                     |
| Convergencia           | Lenta; sufre la cuenta a infinito      | <b>Rápida</b> , sin cuenta a infinito              |
| Recursos               | Pocos                                  | Más memoria y procesamiento                        |

## 3. Protocolo IP (Tanenbaum 5.6.1)

**Header IPv4: 20 bytes fijos + hasta 40 de opciones (en filas de 32 bits)**

|                                       |   |           |    |                           |    |
|---------------------------------------|---|-----------|----|---------------------------|----|
| 0                                     | 4 | 8         | 16 | 19                        | 31 |
| +-----+-----+-----+-----+-----+-----+ |   |           |    |                           |    |
| VERS                                  |   | HLEN      |    | Tipo Servicio             |    |
|                                       |   |           |    | Longitud total            |    |
| +-----+-----+-----+-----+-----+-----+ |   |           |    |                           |    |
| Identificación                        |   |           |    | Flags  Desplaz. fragmento |    |
| +-----+-----+-----+-----+-----+-----+ |   |           |    |                           |    |
| TTL                                   |   | Protocolo |    | Checksum de cabecera      |    |
| +-----+-----+-----+-----+-----+-----+ |   |           |    |                           |    |
| Dirección IP origen                   |   |           |    |                           |    |
| Dirección IP destino                  |   |           |    |                           |    |
| Opciones (si hay)                     |   |           |    | Relleno                   |    |
| +-----+-----+-----+-----+-----+-----+ |   |           |    |                           |    |

| CAMPO | BITS | PARA QUÉ    |
|-------|------|-------------|
| VERS  | 4    | Versión (4) |

| CAMPO            | BITS   | PARA QUÉ                                                                    |
|------------------|--------|-----------------------------------------------------------------------------|
| HLEN             | 4      | Largo del header en palabras de 32 bits: mínimo 5 (20 B), máximo 15 (60 B)  |
| Tipo de Servicio | 8      | <b>Prioridad (3 bits)</b> + bits <b>D, T, R</b> + 2 sin uso (detalle abajo) |
| Longitud total   | 16     | Header + datos en bytes; máximo <b>65.535</b>                               |
| Identificación   | 16     | Identifica el datagrama; <b>los fragmentos comparten este valor</b>         |
| Flags            | 3      | 1 sin uso + <b>DF</b> (no fragmentar) + <b>MF</b> (más fragmentos)          |
| Desplazamiento   | 13     | Posición del fragmento, <b>en unidades de 8 bytes</b>                       |
| TTL              | 8      | –1 por cada router; en 0 se descarta y se avisa con <b>ICMP 11</b>          |
| Protocolo        | 8      | <b>1 ICMP · 2 IGMP · 6 TCP · 17 UDP</b>                                     |
| Checksum         | 16     | Solo del <b>header</b> ; se recalcula en cada salto (porque cambia el TTL)  |
| Origen / Destino | 32 c/u | Direcciones IP                                                              |

### Tipo de Servicio, versión de Baró (RFC 791):

- **Prioridad / Precedencia** (3 bits, de 0 a 7): **es el subcampo que indica la prioridad del datagrama.**
- **D (Delay)**: bajo retardo ("enviar rápido").
- **T (Throughput)**: **alto rendimiento**, el mayor volumen en el menor tiempo ("enviar mucho").
- **R (Reliability)**: alta confiabilidad, que no se pierda ni se dañe ("enviar bien").
- Son **sugerencias**: cada router puede respetarlas o ignorarlas.
- *Hoy ese byte se llama DiffServ (6 bits) + ECN (2 bits), que es como lo presentó Medín. Para Baró contestá con la versión ToS.*

### Opciones (pensadas para pruebas y depuración):

- **Seguridad**: indica qué tan secreto es el datagrama.
- **Ruteo estricto desde el origen** (*strict source routing*): el emisor fija el camino completo.
- **Ruteo libre desde el origen** (*loose source routing*): el emisor fija routers por los que tiene que pasar sí o sí.
- **Registro de ruta**: cada router por el que pasa **anota su IP**. Sirve para saber **qué camino siguió el datagrama** y depurar el ruteo; entran unas 9 direcciones.
- **Marca de tiempo**: como el registro de ruta, pero cada router anota también **la hora** (32 bits). Sirve para medir demoras por tramo.

### Fragmentación

- La **MTU** es el máximo de datos que entra en una trama de una red: Ethernet **1500 B**. Si el próximo tramo tiene una MTU menor que el datagrama, **el router lo fragmenta.**
- Cada fragmento es un datagrama con la **misma Identificación**, su propio **desplazamiento** y **MF = 1**, salvo el último, que lleva **MF = 0**. Los datos de cada fragmento, menos el último, tienen que ser **múltiplo de 8**.
- **Solo el destino reensambla**; los routers intermedios no.
- Si **DF = 1** y hace falta fragmentar, el datagrama **se descarta.**
- **Receta**: datos por fragmento =  $MTU - 20$ , redondeado hacia abajo a múltiplo de 8. Campo desplazamiento =  $(\text{byte de inicio del fragmento}) / 8$ .

Ejemplo (apunte): 1400 B de datos, con MTU 620 en el tramo siguiente.

| Frag | Long.total | Datos     | Desplaz. (bytes) | Campo (÷8) | MF |
|------|------------|-----------|------------------|------------|----|
| 1    | 620        | 0–599     | 0                | 0          | 1  |
| 2    | 620        | 600–1199  | 600              | 75         | 1  |
| 3    | 220        | 1200–1399 | 1200             | 150        | 0  |

**ATENCIÓN** El apunte escribe el desplazamiento en **bytes**. El valor real del campo es ese número **dividido 8**.

## 4. Direcciones IP y subnetting (Tanenbaum 5.6.2) – el núcleo práctico

### Clases y máscaras por defecto

| CLASE | PRIMEROS BITS | 1ER BYTE | FORMATO    | MÁSCARA             | HOSTS POR RED |
|-------|---------------|----------|------------|---------------------|---------------|
| A     | 0             | 0–127    | r.h.h.h    | 255.0.0.0 (/8)      | 16.777.214    |
| B     | 10            | 128–191  | r.r.h.h    | 255.255.0.0 (/16)   | 65.534        |
| C     | 110           | 192–223  | r.r.r.h    | 255.255.255.0 (/24) | 254           |
| D     | 1110          | 224–239  | multicast  | —                   | —             |
| E     | 1111          | 240–255  | reservadas | —                   | —             |

**Privadas:** A 10.0.0.0, B 172.16.0.0 – 172.31.0.0, C 192.168.0.0 – 192.168.255.0. Para qué existen:

1. Permiten armar redes internas **sin contratar IPs públicas**.
2. **No chocan con Internet**, porque nunca se asignan a hosts públicos.
3. **Cualquier organización puede reusarlas**, y eso ahorra direcciones IPv4.
4. **Desde afuera no se llega a ellas**: salen por un router, proxy o NAT con IP pública, y eso funciona como protección.

### Direcciones especiales:

| DIRECCIÓN       | SIGNIFICADO                                                        |
|-----------------|--------------------------------------------------------------------|
| 0.0.0.0         | Este host (se usa al bootear)                                      |
| 0.0.0.N         | El host N de <b>mi</b> red                                         |
| red + host en 0 | <b>La red</b> (no se asigna a ningún host)                         |
| 255.255.255.255 | Broadcast a <b>mi</b> red                                          |
| red + host en 1 | Broadcast a <b>esa</b> red                                         |
| 127.x.x.x       | <b>Loopback</b> : prueba la pila TCP/IP propia, no la placa de red |

## Método (lo que se calcula en el parcial)

```
Red          = IP AND máscara
Broadcast    = IP OR (NOT máscara)      (= red con todos los bits de host en 1)
Hosts        = 2^(bits de host) - 2      (se restan la de red y la de broadcast)
1er host     = red + 1                  último host = broadcast - 1
Salto        = 256 - (byte "interesante" de la máscara)
```

### Paso a paso:

1. Ubicá el **byte interesante**, el que en la máscara no es ni 0 ni 255. Los bytes con 255 se copian de la IP; los que tienen 0 dan 0 en la red y 255 en el broadcast.
2. Calculá el **salto** = 256 - byte de la máscara. Las subredes arrancan en **múltiplos del salto**.
3. La **red** es el múltiplo del salto  $\leq$  byte de la IP; el **broadcast** es el múltiplo siguiente - 1.
4. Si dudás, pasá a binario solo ese byte y hacé el AND.

**Ejemplo:** 192.168.20.25 / 255.255.255.240. El salto es  $256 - 240 = 16$ , así que las subredes son .0, .16, .32... El 25 cae en [16, 32), entonces la red es **192.168.20.16** y el broadcast **192.168.20.31**, con 14 hosts (.17 a .30).

### Subredes de una clase C:

| Máscara         | /   | Subredes | Hosts | Salto                     |
|-----------------|-----|----------|-------|---------------------------|
| 255.255.255.128 | /25 | 2        | 126   | 128                       |
| 255.255.255.192 | /26 | 4        | 62    | 64                        |
| 255.255.255.224 | /27 | 8        | 30    | 32                        |
| 255.255.255.240 | /28 | 16       | 14    | 16                        |
| 255.255.255.248 | /29 | 32       | 6     | 8                         |
| 255.255.255.252 | /30 | 64       | 2     | 4 (enlaces punto a punto) |

### Para elegir la máscara:

- **Por cantidad de subredes:** tomá  $n$  bits prestados tal que  $2^n \geq$  subredes.
- **Por cantidad de hosts:** dejá  $h$  bits de host tal que  $2^h - 2 \geq$  hosts.

**Problema inverso** (te dan el rango de hosts y piden la red): pasá a binario el primero y el último. Los **bits en común** son la red y el resto es el host. Ejemplo: .145 a .158  $\rightarrow$  1001 0001 / 1001 1110  $\rightarrow$  4 bits en común  $\rightarrow$  red .144, broadcast .159, máscara 255.255.255.240.

**Caso práctico de diseño (apunte):** una empresa tiene la pública 194.143.17.8/29 (red .8, broadcast .15, hosts .9-.14) y necesita 3 servidores y 20 PCs.

- **Red pública:** router .9 (gateway de los servidores), **proxy .10**, web y correo en las otras públicas.
- **Red privada:** las 20 PCs en 192.168.1.0/24 con **gateway 192.168.1.1**, que es el proxy.
- **El proxy tiene dos IPs**, una por red: deja salir a la LAN y bloquea las entradas desde afuera.
- **La idea:** solo se gasta IP pública en lo que tiene que verse desde Internet.

**Cómo decide un host:** aplica su máscara a la IP destino. Si da su misma red, **entrega directo** (ARP al destino). Si no, se lo manda al **gateway** (ARP al gateway).

## 5. Protocolos de control (Tanenbaum 5.6.4) y comandos de red

### ICMP – capa 3

- **Informa errores y transporta mensajes de control.** No decide nada: eso queda para las capas superiores. Viaja **dentro de un datagrama IP** (Protocolo = 1), pero es de **capa 3 (red)**.
- Si un mensaje ICMP se pierde, **no se genera otro ICMP** por eso.
- También es el protocolo de control que se usa con las opciones **registro de ruta** y **marca de tiempo** (ping -r, tipos 13/14).

|                              |                                        |
|------------------------------|----------------------------------------|
| 0 Respuesta de eco (ping)    | 8 Solicitud de eco (ping)              |
| 3 Destino inaccesible        | 11 Tiempo excedido (TTL=0) → tracert   |
| 4 Source quench (en desuso)  | 12 Problema de parámetros              |
| 5 Redireccionar (mejor ruta) | 13/14 Marca de tiempo    17/18 Máscara |

**PING** manda un tipo 8 y recibe un tipo 0. Prueba las capas **física, de enlace y de red**, pero **no** transporte ni aplicación.

- **Respuesta:** todo bien hasta capa 3.
- **Tiempo agotado:** revisar el host destino y el cableado. Si hay un router en el medio, hacé ping al gateway primero.
- **Host inaccesible** (ICMP 3): IP o máscara mal (no están en la misma red), o el gateway mal configurado.
- **Error:** la pila TCP/IP del host está mal. Se prueba con `ping 127.0.0.1`.
- **TRACERT / traceroute** manda paquetes con **TTL = 1, 2, 3...** (hasta 30). Cada router que lleva el TTL a 0 devuelve un **ICMP 11** con su IP, y así se arma el camino salto a salto. **Windows usa ICMP echo** y **Unix usa UDP**.

### ARP – de IP a MAC

- **Para qué** (pregunta de la guía): para conocer la **MAC** que corresponde a una IP **dentro de la LAN**. La trama Ethernet necesita la MAC de destino y el datagrama solo trae la IP. Si el destino está en otra red, se averigua la MAC del **gateway**.
- **Cómo funciona:** la pregunta va en **broadcast** ("¿quién tiene la IP X?") y lleva la IP y la MAC de quien pregunta. La respuesta vuelve **directa** (unicast).
- **Caché ARP:** guarda los pares IP↔MAC con un **tiempo de vida**, para no preguntar de nuevo. Como todos escuchan la pregunta, todos pueden anotar a quien preguntó.
- **Las IP de origen y destino no cambian en todo el viaje; las MAC cambian en cada red.** Es el mismo datagrama viajando en tramas distintas.
- **Trama Ethernet:** preámbulo 8 · MAC destino 6 · MAC origen 6 · tipo 2 · datos 46–1500 · CRC 4 (en bytes). *El apunte dice datos de 64–1500, pero 64 B es el mínimo de la trama entera; el mínimo de datos es 46.*

### DHCP y BOOTP

- **DHCP:** al arrancar, el host pide configuración por **broadcast**. El servidor le asigna IP con un **tiempo de arriendo** y además le pasa **máscara, gateway y DNS**.
- **BOOTP** es su antecesor. [Conocimiento general] Asignaba la configuración al bootear desde una tabla **fija** que cargaba el administrador, sin arriendo. DHCP lo reemplazó (junto con RARP).

## 6. OSPF y BGP (Tanenbaum 5.6.6-5.6.7)

---

### Sistemas Autónomos e IGP

**Sistema Autónomo (SA) [Tanenbaum]:** una red o conjunto de redes **bajo una única administración** (un ISP, una empresa, una universidad), con **su propia política de ruteo**. Se identifica con un **número de SA**.

- **Cómo está conformado:** **routers internos**, que rutean dentro del SA con un **IGP**, y **routers frontera** (*border*), que lo conectan con otros SA usando **BGP**. Internet es una interconexión de SA.

**IGP (Interior Gateway Protocol) [Tanenbaum]:** protocolo de ruteo **intradominio**, que trabaja **dentro de un SA**.

- **Función:** calcular las rutas **más eficientes** (de menor costo) entre las redes del SA y adaptarse rápido a los cambios. Como hay un solo administrador, **no hay políticas**: solo importa la eficiencia.
- **Ejemplos:** RIP, OSPF, IS-IS.
- Su contraparte entre SA es el **EGP**, protocolo **interdominio**: **BGP**.
- **RIP:** vector distancia, **IGP** (dentro de un SA), con la **cantidad de saltos** como métrica. Sirve para **redes chicas**: 16 saltos = infinito [*conocimiento general*]. Sufre la cuenta a infinito, por eso OSPF lo reemplazó en redes medianas y grandes.

### OSPF (Open Shortest Path First)

#### Principios:

- Es un IGP de **estado de enlace**, con los 5 pasos de la sección 2 y Dijkstra. **Abierto**: estándar publicado (RFC 2328).
- Admite **varias métricas** (distancia, retardo...).
- Se **adapta rápido** a los cambios de topología.
- Hace **balanceo de carga** entre caminos de igual costo (**ECMP**).
- Soporta **jerarquía en áreas** y **autenticación** de los mensajes.
- Mensajes: **Hello** (descubrir vecinos) y **Link State Update** (enviar el estado). [*Tanenbaum*] También Link State Ack, Database Description y Link State Request.
- En cada LAN se elige un **router designado** (más un backup) que habla por todos; así no hace falta que cada par de routers se comuniquen.

#### Ventajas sobre vector distancia:

- **Converge rápido y no tiene cuenta a infinito.**
- **Cada router conoce la topología completa.**
- Mejores métricas: costo por ancho de banda, en vez de solo saltos.
- Hace **balanceo de carga**.
- **Escala** gracias a las áreas.
- Solo envía **cambios**, no la tabla entera.
- **Qué quiere decir que reconoce jerarquías [Tanenbaum]:** el SA se divide en **áreas**, y **cada router conoce en detalle solo la topología de su área**. Del resto conoce solo resúmenes. El tráfico entre áreas **pasa siempre por el backbone**: área origen → router de borde → backbone → router de borde → área destino. Con esto se achican las tablas y los cálculos.

**Área y tipos de área [Tanenbaum]:** un área es una **parte del SA**, una red o un grupo de redes contiguas.

- **Backbone (área 0):** las conecta a todas. Toda área tiene que estar conectada al backbone.
- **Stub (área terminal):** tiene **una sola salida**, así que no recibe las rutas externas y usa una **ruta default**.
- **Áreas comunes:** las demás.
- **Routers [Tanenbaum]:** **internos** (todo dentro de un área), **de borde de área** (conectan su área con el backbone), **del backbone** y **de frontera del SA** (hablan con otros SA).

### BGP (Border Gateway Protocol)

- **En qué consiste:** es el protocolo de ruteo **interdominio, entre Sistemas Autónomos**. Lo corren los **routers frontera**, que establecen **conexiones TCP** entre sí.
- **Entorno:** Internet a nivel de SA (ISP con ISP, empresas *multihomed* con sus proveedores).
- **Filosofía: ruteo por políticas (policy-based).** No importa solo la ruta más corta, sino **qué tráfico se acepta llevar y para quién**, por razones económicas, de seguridad o políticas. Por ejemplo, no hacer de tránsito gratis para un competidor. Se distinguen relaciones **cliente-proveedor** (se paga el tránsito) y **peering** (tráfico recíproco gratis).
- **Algoritmo: vector de ruta (path vector).** Cada router anuncia la **ruta completa**, la lista de SA que atraviesa, y no solo la distancia. Así **detecta bucles** (descarta toda ruta que ya lo contiene) y **evita la cuenta a infinito**.

## 7. Guía de estudio de Baró – preguntas y respuestas

Las preguntas son **textuales** de la guía de Baró ( GUIA DE ESTUDIO DE CAPA DE RED.docx ). Lo marcado **[Tanenbaum]** no está en tus fuentes: lo completé con el contenido del libro.

### Protocolo IP - Direcciones IP - Subnetting

**P1. ¿Qué campo/subcampo de la Cabecera IP indica la prioridad del datagrama?**

**R:** El subcampo **Prioridad** (o Precedencia), que ocupa los **3 primeros bits del campo Tipo de Servicio**. Va de **0** (prioridad baja) a **7** (prioridad máxima).

**P2. El indicador "T" dentro del campo Tipo de Servicio del Datagrama IP tiene una finalidad específica, ¿cuál es?**

**R:** El bit **T (Throughput)** le pide a los routers **alto rendimiento**: que el datagrama vaya por la ruta que permita transmitir **el mayor volumen de datos en el menor tiempo** ("enviar mucho"). Es una **sugerencia**: cada router puede respetarla o ignorarla. Junto con él están **D (Delay**: bajo retardo, "enviar rápido") y **R (Reliability**: alta confiabilidad, "enviar bien").

**P3. Dentro del Campo de Opciones del datagrama IP existe una opción que se utiliza como "registro de ruta", ¿cuál es su utilidad?**

**R:** Cada router por el que pasa el datagrama **agrega su dirección IP** en esa opción. Así se puede saber **qué camino siguió el datagrama**, lo que sirve para **diagnosticar y depurar problemas de ruteo** (por ejemplo, detectar un desvío o un bucle). Como el espacio de opciones es de 40 bytes, entran unas **9 direcciones**.

**P4. ¿Cuál es la función/utilidad de definir rangos de direcciones privadas dentro de cada clase de direcciones IP?**

**R:** Permiten armar **redes internas (intranets)** sin contratar direcciones públicas, que son escasas y caras. Las ventajas son:

- **No hay conflicto con Internet:** esos rangos nunca se asignan a hosts públicos.
- **Se ahorran direcciones IPv4:** todas las organizaciones pueden reusar los mismos rangos.
- **Seguridad:** los hosts privados **no son accesibles desde Internet**. Salen a través de un router, proxy o NAT con IP pública, pero nadie puede entrar a ellos desde afuera.

Los rangos son: clase A **10.0.0.0**, clase B **172.16.0.0** a **172.31.0.0** y clase C **192.168.0.0** a **192.168.255.0**.

## Protocolos de Control de Internet

### P1. ¿Con qué finalidad se realiza una petición ARP dentro de una LAN?

**R:** Para obtener la **dirección física (MAC)** que corresponde a una dirección IP. El datagrama solo trae la IP destino, pero para enviarlo por la LAN hay que armar una trama con la **MAC de destino**. Si el destino está en la misma red, se pregunta por su MAC; si está en otra, se pregunta por la MAC del **router/gateway**. La petición va por **broadcast** ("¿quién tiene la IP X?") y lleva la IP y la MAC de quien pregunta. La respuesta vuelve **directa** y se guarda en la **caché ARP**.

### P2. ¿A qué capa del modelo OSI pertenece el protocolo ICMP?

**R:** A la **capa 3 (red)**. Sus mensajes viajan **encapsulados dentro de datagramas IP** (campo Protocolo = 1), pero eso no lo convierte en un protocolo de capa 4: es el protocolo de control y error que acompaña a IP.

### P3. Cuando se utilizan las opciones de cabecera "Registro de Ruta" y "Marca de Tiempo", ¿qué protocolo de control se utiliza?

**R: ICMP.** Las dos opciones se usan con los mensajes de **eco** de ICMP (**ping -r** hace un ping con registro de ruta), y ICMP tiene además los mensajes específicos de **marca de tiempo** (tipos **13**, solicitud, y **14**, respuesta).

## Protocolos de Ruteo

### P1. Defina el concepto de Ruteo en Internet.

**R:** Es el proceso por el cual los **routers deciden por qué línea de salida reenviar cada datagrama** para que llegue a la red destino. Para eso usan una **tabla de ruteo**, que arman y actualizan con **algoritmos y protocolos de ruteo**: un IGP dentro de cada Sistema Autónomo y BGP entre Sistemas Autónomos. Se distingue el **reenvío** (consultar la tabla para cada paquete) del **ruteo** propiamente dicho (armar la tabla). [Tanenbaum]

### P2. ¿En qué consiste una Tabla de Ruteo? ¿Quién la arma?

**R:** Es la tabla del router que asocia cada **red destino** (prefijo) con la **línea de salida o próximo salto** por donde hay que mandar el paquete, y con la **métrica** de esa ruta. La arma **cada router**: dinámicamente, ejecutando un protocolo de ruteo (RIP, OSPF, BGP) que intercambia información con los demás routers, o el **administrador a mano**, si las rutas son estáticas. [Tanenbaum]

### P3. ¿Qué es una ruta default? ¿Por qué motivo existe dicha entrada dentro de la Tabla de Ruteo?

**R:** Es la entrada que se usa **cuando la red destino no coincide con ninguna otra entrada** de la tabla. Se escribe **0.0.0.0/0**. Existe porque **un router no puede tener en su tabla todas las redes de Internet**: lo que no conoce lo manda a un router "de salida" (el del ISP), que sí sabe cómo seguir. Un host hace lo mismo con su **puerta de enlace (gateway)**. [Tanenbaum]

**P4. ¿En qué consiste un Sistema Autónomo? ¿Cómo está conformado? Describa.**

**R:** Es una red o un conjunto de redes **bajo una única administración** (un ISP, una empresa, una universidad), con **su propia política de ruteo**, identificado por un **número de SA**. Internet es la interconexión de miles de Sistemas Autónomos. Está formado por:

- **Routers internos**, que rutean dentro del SA usando un **IGP** (RIP, OSPF).
- **Routers frontera (border)**, que conectan el SA con otros SA usando **BGP**.

[Tanenbaum]

**P5. ¿Qué es un IGP? Funciones que cumple y entorno de aplicación.**

**R:** Un IGP (Interior Gateway Protocol) es un protocolo de ruteo **interior o intradominio**.

- **Funciones:** calcular las **rutas de menor costo** entre las redes del SA, mantener actualizadas las tablas y **adaptarse rápido a los cambios** de topología. Como hay un solo administrador, no maneja políticas: solo busca **eficiencia**.
- **Entorno:** dentro de un Sistema Autónomo.
- **Ejemplos:** RIP, OSPF, IS-IS.

Su contraparte entre Sistemas Autónomos es el protocolo exterior, **BGP**. [Tanenbaum]

**P6. ¿Qué diferencia operativa existe entre un protocolo de Vector Distancia y uno de Estado de Enlace?**

**R:**

- **Vector distancia (RIP):** cada router envía **su tabla completa** (su distancia a todos los destinos) **solo a sus vecinos**, de forma periódica. Ningún router conoce la topología; cada uno recalcula con Bellman-Ford. **Converge lento** y sufre el problema de la **cuenta a infinito**.
- **Estado de enlace (OSPF):** cada router envía **solo el estado de sus propios enlaces** (sus vecinos y el costo a cada uno) **a todos los routers**, por inundación. Así **todos conocen la topología completa** y cada uno calcula sus rutas con **Dijkstra**. **Converge rápido** y no tiene cuenta a infinito, a cambio de **más memoria y procesamiento**.

**P7. ¿Qué es una Métrica y para qué sirve? Dé ejemplos.**

**R:** Es el **valor de costo** que se le asigna a un enlace o a una ruta. Sirve para **comparar caminos y elegir el mejor** (el de menor costo total). Ejemplos:

- **Cantidad de saltos (RIP).**
- **Retardo.**
- **Ancho de banda:** en OSPF el costo es inversamente proporcional a la velocidad (1 Gbps → 1; 100 Mbps → 10).
- **Carga del enlace, distancia, costo económico.**

**P8. ¿En qué entorno trabaja el protocolo RIP?**

**R: Dentro de un Sistema Autónomo:** es un IGP de **vector distancia**, pensado para **redes chicas**. Usa como métrica la **cantidad de saltos**, con un máximo de **15** (16 se toma como infinito), y por eso no sirve para redes grandes. Por su convergencia lenta fue reemplazado por OSPF en redes medianas y grandes.

**P9. ¿Qué principios de funcionamiento tiene el OSPF? ¿Cuáles son las ventajas sobre los protocolos de Vector Distancia?**

**R:** Principios de funcionamiento:

- Es un IGP de **estado de enlace** y un **estándar abierto** (RFC 2328).
- Descubre a sus vecinos con mensajes **Hello**, mide el costo de cada enlace, **inunda** paquetes de estado de enlace (*Link State Update*) y calcula las rutas con **Dijkstra**.
- Admite **varias métricas** y hace **balanceo de carga** entre caminos de igual costo (**ECMP**).
- Es **jerárquico**: divide el SA en **áreas**.
- **Autentica** los mensajes.
- En cada LAN elige un **router designado** (más uno de backup) que habla por todos.

Ventajas sobre vector distancia:

- **Converge rápido** y **no tiene cuenta a infinito**.
- Cada router **conoce la topología completa**.
- Usa **mejores métricas** (ancho de banda, no solo saltos).
- **Escala** a redes grandes gracias a las áreas.
- Solo envía **cambios**, no la tabla entera.

**P10. ¿Qué quiere decir que OSPF reconoce jerarquías de ruteo?**

**R:** Que divide el Sistema Autónomo en **áreas** conectadas a un **área backbone (área 0)**, en dos niveles. **Cada router conoce en detalle solo la topología de su área**; del resto conoce solo resúmenes. El tráfico entre áreas **siempre pasa por el backbone**: área origen → router de borde → backbone → router de borde → área destino. Así se **achican las tablas, los mensajes y el cálculo** de cada router. [Tanenbaum]

**P11. ¿Qué es un área dentro de un Sistema Autónomo y qué tipos de áreas existen?**

**R:** Un área es una **porción del SA**: una red o un grupo de redes contiguas con su propia topología interna, que los routers de otras áreas no ven en detalle. Tipos:

- **Backbone (área 0)**: el área central que **conecta a todas las demás**. Toda área tiene que estar conectada al backbone.
- **Área stub (terminal)**: tiene **una sola salida**, por eso no recibe las rutas externas y usa una **ruta default** para salir.
- **Áreas comunes**: las demás, conectadas al backbone.

Según su ubicación, los routers pueden ser **internos, de borde de área, del backbone o de frontera del SA**. [Tanenbaum]

**P12. ¿En qué consiste el protocolo BGP? ¿En qué entorno se aplica? ¿Qué filosofía de ruteo utiliza?**

**R:**

- **En qué consiste**: es el protocolo de ruteo **exterior o interdominio** (*Border Gateway Protocol*). Lo corren los **routers frontera**, que intercambian rutas sobre **conexiones TCP**.
- **Entorno**: **entre Sistemas Autónomos**, es decir, Internet a nivel de ISP y de organizaciones conectadas a uno o varios proveedores (*multihoming*).
- **Filosofía**: **ruteo por políticas**. No busca solo la ruta más corta, sino decidir **qué tráfico acepta llevar y para quién**, por razones económicas, de seguridad o políticas. Por ejemplo, no ser tránsito gratis de un competi-

dor. Se distinguen las relaciones **cliente-proveedor** (se paga el tránsito) y **peering** (intercambio recíproco gratis).

- **Algoritmo: vector de ruta** (*path vector*). Cada router anuncia la **ruta completa** (la lista de SA que atraviesa); así **detecta bucles** y evita la cuenta a infinito.

### P13. ¿Cómo están compuestos los paquetes de Estado de Enlace en OSPF?

R: Cada paquete de estado de enlace (LSP) contiene:

- La **identidad del router emisor**.
- Un **número de secuencia**, para descartar duplicados y paquetes viejos.
- Una **edad**, que se va decrementando; el paquete se descarta al llegar a 0.
- La **lista de sus vecinos con el costo (métrica) a cada uno**.

[Tanenbaum]

### P14. ¿En qué consiste un Grafo en OSPF y con qué fines se realiza?

R: Con los paquetes de estado de enlace que recibe de todos los routers, **cada router arma un grafo de la topología completa**: los **nodos** son los routers y las **aristas** son los enlaces, con su costo. Ese grafo se arma para **calcular sobre él, con Dijkstra, la ruta más corta a cada destino** y así llenar la tabla de ruteo. Cada router lo calcula **localmente** y con información completa. [Tanenbaum]

### P15. ¿Cuáles son los 5 pasos que ejecuta el OSPF para aprender y difundir rutas óptimas?

R:

1. **Descubrir a sus vecinos** y conocer sus direcciones (paquetes Hello).
2. **Medir el costo** (la métrica) del enlace a cada vecino.
3. **Armar un paquete de estado de enlace** con todo lo aprendido.
4. **Enviar ese paquete a todos los routers** por inundación, y recibir los de todos los demás.
5. **Calcular la ruta más corta** a cada router con **Dijkstra**.

## 8. Ejercicios para practicar (con respuestas)

Intentalos antes de mirar la respuesta. Están verificados con Python (módulo `ipaddress`).

**E1.** 172.16.45.200 / 255.255.240.0. ¿Red, broadcast, rango de hosts y cantidad? **E2.** 10.25.130.7 / 255.255.255.192. ¿Red y broadcast? **E3.** ¿Están 192.168.1.70 y 192.168.1.130 en la misma subred con máscara /25? **E4.** Hay que dividir 192.168.10.0/24 en **6 subredes**. ¿Qué máscara usás? Listá las subredes. **E5.** Desde 200.10.5.0/24 se necesitan subredes de **50 hosts** cada una. ¿Qué máscara usás, cuántas subredes salen y cuáles son? **E6.** Los hosts de una red van de 200.1.1.65 a 200.1.1.94. ¿Red, broadcast y máscara? **E7.** ¿De qué clase es cada una? ¿Es pública o privada? 130.5.2.1 · 223.1.1.1 · 10.0.0.1 · 172.32.1.1 · 172.20.1.1 **E8.** Un datagrama de **4000 B en total** (header de 20 B) tiene que cruzar una Ethernet (MTU 1500). Armá los fragmentos con su longitud total, el valor del campo desplazamiento y MF. **E9.** Hacés ping a un host de otra red y te devuelve "Host de destino inaccesible" desde 192.168.0.1. ¿Qué mensaje ICMP es, quién lo genera y qué revisás?

### Respuestas

- **E1.** Salto  $256 - 240 = 16$  en el 3er byte, y 45 cae en  $[32, 48)$ . Red **172.16.32.0**, broadcast **172.16.47.255**, hosts .32.1 a .47.254: **4094** ( $2^{12} - 2$ ). Es una /20.
- **E2.** Salto 64 en el 4to byte, y 7 cae en  $[0, 64)$ . Red **10.25.130.0**, broadcast **10.25.130.63**, 62 hosts.
- **E3. No.** Con /25 (salto 128), 70 cae en la subred .0 y 130 en la subred .128.
- **E4.** Hace falta  $2^n \geq 6$ , entonces  $n = 3$ : /27 (**255.255.255.224**). Salen 8 subredes de 30 hosts: .0, .32, .64, .96, .128, .160, .192 y .224 (sobran 2).
- **E5.** Hace falta  $2^h - 2 \geq 50$ , entonces  $h = 6$ : /26 (**255.255.255.192**). Salen **4 subredes** de 62 hosts: .0, .64, .128 y .192.
- **E6.**  $65 = 0100\ 0001$  y  $94 = 0101\ 1110$ : 3 bits en común. Red **200.1.1.64**, broadcast **200.1.1.95**, máscara **255.255.255.224** (/27).
- **E7.** 130.5.2.1 es **B pública**; 223.1.1.1 es **C pública**; 10.0.0.1 es **A privada**; 172.32.1.1 es **B pública** (queda fuera de 172.16–172.31, una trampa típica); 172.20.1.1 es **B privada**.
- **E8.** Hay 3980 B de datos y entran 1480 por fragmento ( $1500 - 20$ , que ya es múltiplo de 8).

| Frag | Long.total | Datos | Campo desplaz. | MF |
|------|------------|-------|----------------|----|
| 1    | 1500       | 1480  | 0              | 1  |
| 2    | 1500       | 1480  | 185 (1480/8)   | 1  |
| 3    | 1040       | 1020  | 370 (2960/8)   | 0  |

Todos los fragmentos llevan la misma Identificación y reensambla solo el destino.

- **E9.** Es un **ICMP tipo 3, "destino inaccesible"**, y lo genera el **gateway** (192.168.0.1) porque no tiene camino hacia esa red. Revisá la configuración IP, la máscara y el gateway del host, y la tabla del router.

## Datos de memoria

- Header IPv4: **20 B** mínimo, **60 B** máximo. Longitud total máxima **65.535**. TTL máximo **255**. Desplazamiento en **unidades de 8 B**.
- Campo Protocolo: **1 ICMP · 2 IGMP · 6 TCP · 17 UDP**.
- Tipos ICMP: **0/8** eco · **3** inaccesible · **4** source quench · **5** redirect · **11** tiempo excedido · **12** parámetro · **13/14** marca de tiempo.
- Rangos privados: **10/8 · 172.16/12 · 192.168/16**. Loopback **127.0.0.1**.
- MTU Ethernet **1500**. Trama Ethernet: MAC de **6 B** cada una.
- **RIP** = vector distancia, saltos, IGP · **OSPF** = estado de enlace, Dijkstra, áreas, IGP · **BGP** = vector de ruta, políticas, entre SA, sobre TCP.
- **tracert** (Windows) usa **ICMP**; **traceroute** (Unix) usa **UDP**. Los dos se basan en el **TTL** y en la respuesta **ICMP 11**.